EX.No: 10 **Medical Patient Image Diagnostic & Machine Learning Platform**

DATE: *(MedAI CadX – Medical Image Analysis, Radiomic Feature Extraction and Multi-Model Classification)*

Problem Statement:

Medical image interpretation can require several sequential tasks such as image standardization, noise reduction, contrast enhancement, tissue segmentation, feature extraction and diagnostic classification. Performing these steps separately can make the workflow slow and difficult to demonstrate, compare and evaluate in an academic laboratory environment.

This project implements a web-based medical image diagnostic and machine-learning platform named MedAI CadX. The interface accepts a patient scan with demographic information, applies an image-preprocessing pipeline, extracts quantitative radiomic features, and evaluates multiple supervised machine-learning algorithms. The system also presents diagnostic performance using accuracy, precision, sensitivity/recall, specificity, F1-score, ROC-AUC and a confusion matrix.

Objectives:

1. To provide a web interface for entering patient demographics and uploading a clinical scan image.
2. To standardize the input image and apply Gaussian denoising, CLAHE contrast enhancement, Otsu-based segmentation and Canny edge detection.
3. To extract quantitative statistical, texture and morphological radiomic features from the processed image.
4. To train and compare five supervised machine-learning models: Decision Tree, K-Nearest Neighbors, Logistic Regression, Random Forest and Support Vector Machine.
5. To evaluate the models using train/test splitting, stratified cross-validation, classification metrics, ROC-AUC curves and a confusion matrix.
6. To present the complete workflow in both an all-in-one frame view and a step-by-step diagnostic view.

Software / Hardware Required:

- Operating System: Windows (project shown running in a desktop IDE environment)
- IDE: Antigravity IDE / VS Code-style project workspace
- Language: Python 3.x
- Backend / Web Application: Python web application (project entry file: app.py)
- Machine Learning: scikit-learn-compatible supervised classification workflow
- Image Processing: OpenCV / NumPy-style preprocessing and feature extraction
- Frontend: HTML, CSS and JavaScript (static/css/style.css and static/js/app.js)
- Dataset: Medical image dataset containing normal and pathological patient scans
- Hardware: Standard PC or laptop; GPU is not required for the classical ML workflow shown

Project Directory Structure:

The uploaded project workspace shows the application organized into dataset, model, sample-data, static, template, upload and report folders, together with separate Python modules for preprocessing, feature extraction, model training and report generation.

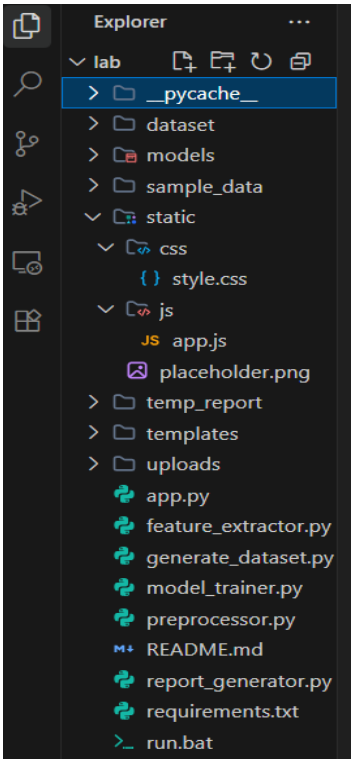


Fig 1: Project directory structure of the MedAI CadX application in the development IDE

Main Project Files:

- app.py – main application entry point.
- preprocessor.py – image standardization and preprocessing operations.
- feature_extractor.py – radiomic / quantitative feature extraction.
- model_trainer.py – training and evaluation of multiple machine-learning models.
- generate_dataset.py – dataset preparation / generation utility.
- report_generator.py – diagnostic report generation utility.
- static/css/style.css – user-interface styling.
- static/js/app.js – front-end interaction logic.
- templates/ – HTML templates used by the web application.
- models/ – saved trained machine-learning models.
- uploads/ – uploaded patient scan images.

Procedure:

1. Launch the MedAI CadX application from the project environment and verify that the trained models are available.
2. Enter the patient ID, patient full name, age, gender and examination/modality information.
3. Upload a patient scan image or choose a benchmark clinical sample.
4. Standardize the scan to a fixed 256×256 image representation.
5. Apply Gaussian filtering to reduce high-frequency noise.
6. Apply CLAHE contrast enhancement to improve local tissue contrast.
7. Create an Otsu ROI mask and perform morphological processing to isolate candidate tissue / lesion structures.
8. Generate a Canny edge map to highlight anatomical boundaries and form the diagnostic pathology overlay.
9. Extract the radiomic feature vector, including first-order statistics, GLCM texture measurements and morphological descriptors.
10. Train / evaluate Decision Tree, KNN, Logistic Regression, Random Forest and SVM models using the selected train/test split and stratified cross-validation.
11. Compare model performance using accuracy, precision, sensitivity/recall, specificity, F1-score and ROC-AUC.
12. Inspect the selected model's confusion matrix and ROC-AUC performance curve.

Representative Implementation Code:

The exact source code was not included in the screenshots. The following short code fragments are representative of the visible project structure and workflow and can be adapted to the corresponding Python modules.

1. Image Preprocessing (preprocessor.py)

```
import cv2
def preprocess_image(img):
    img = cv2.resize(img, (256, 256))
    denoised = cv2.GaussianBlur(img, (5, 5), 0)
    gray = cv2.cvtColor(denoised, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    _, mask = cv2.threshold(
        enhanced, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )
    edges = cv2.Canny(enhanced, 50, 150)
    return denoised, enhanced, mask, edges
```

2. Model Benchmarking (model_trainer.py) import cv2

```
def preprocess_image(img):
    img = cv2.resize(img, (256, 256))
    denoised = cv2.GaussianBlur(img, (5, 5), 0)
    gray = cv2.cvtColor(denoised, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    _, mask = cv2.threshold(
        enhanced, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )
    edges = cv2.Canny(enhanced, 50, 150)
    return denoised, enhanced, mask, edges
```

Output (Step-by-Step Screenshots):

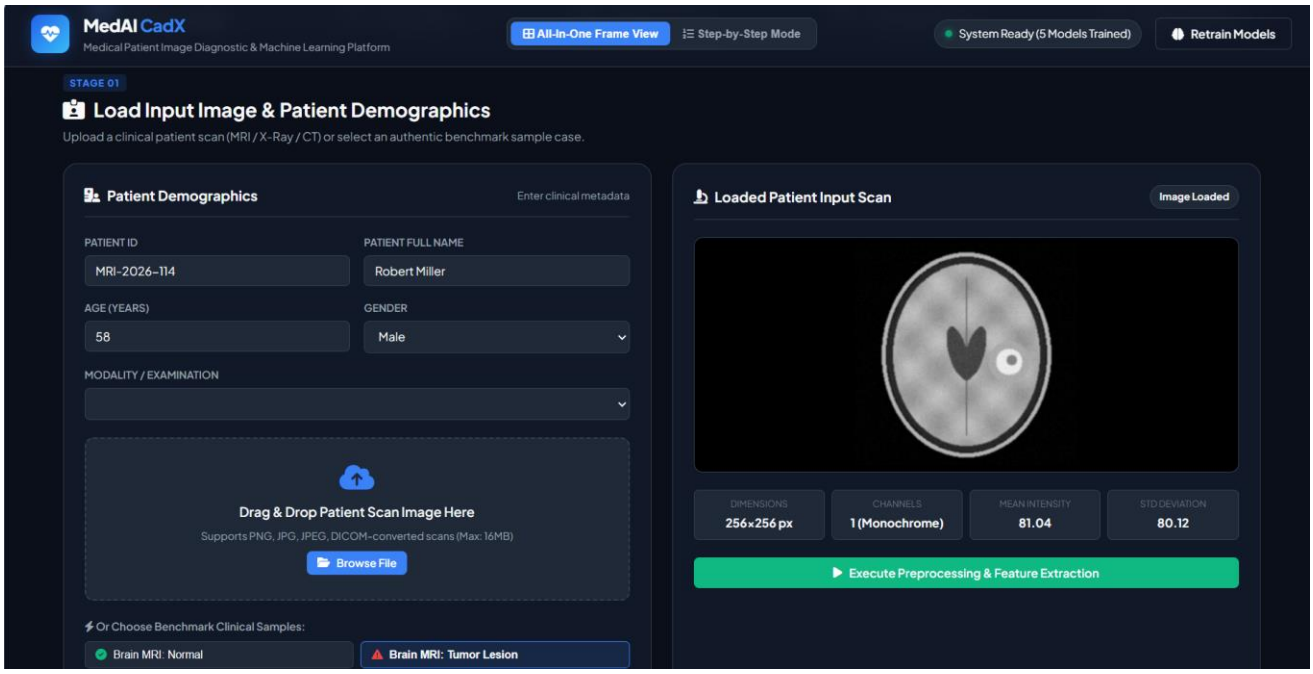


Fig 2: Stage 1 — Load input image and patient demographics

The first stage records patient information and loads a clinical scan. The displayed example contains a 256×256 monochrome brain MRI image and provides a control to execute preprocessing and feature extraction.

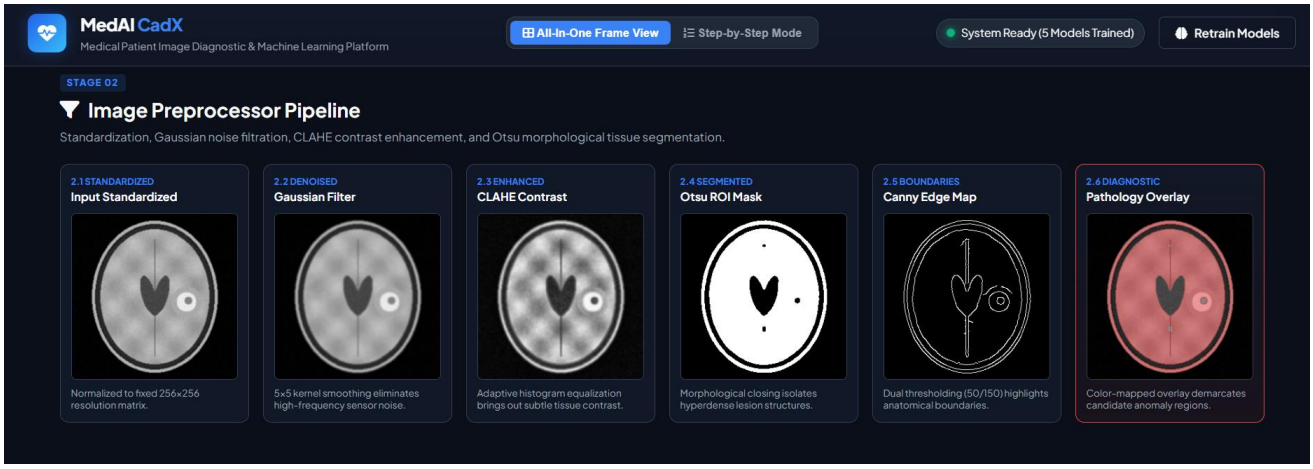


Fig 3: Stage 2 — Image Preprocessor Pipeline

The preprocessing pipeline visibly includes image standardization, Gaussian filtering, CLAHE contrast enhancement, Otsu ROI segmentation, Canny boundary detection and a pathology overlay for candidate abnormal regions.

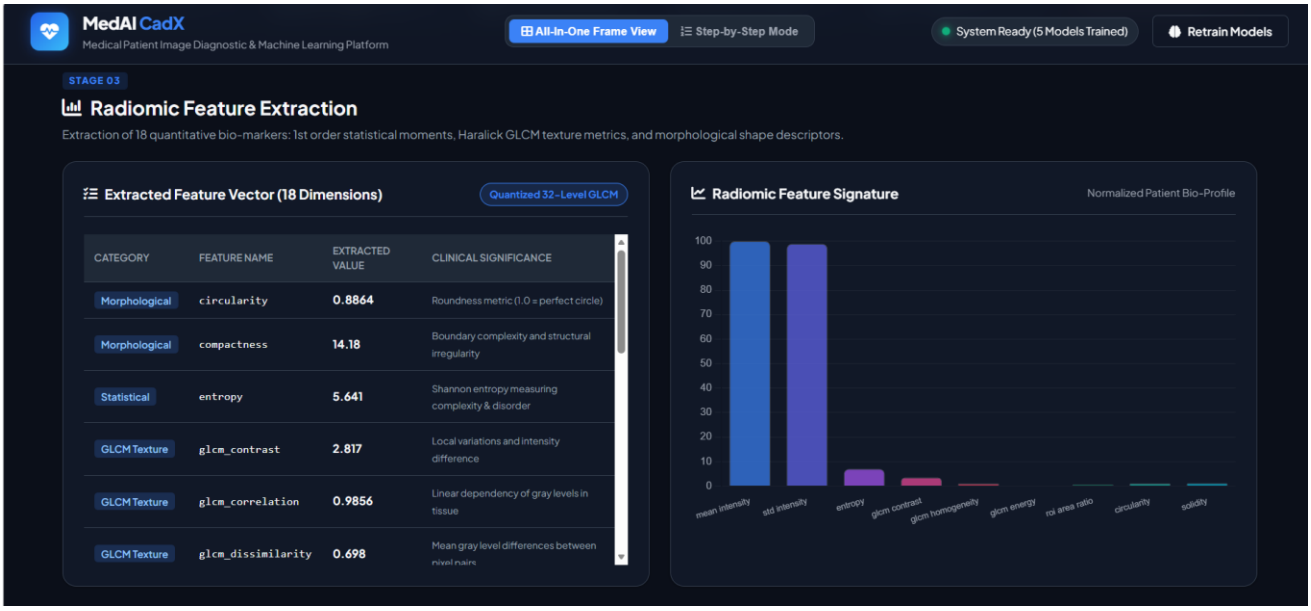


Fig 4: Stage 3 — Radiomic Feature Extraction

The feature-extraction stage reports an 18-dimensional feature vector. Visible examples include circularity, compactness, entropy, GLCM contrast, GLCM correlation and GLCM dissimilarity. A normalized radiomic feature signature is plotted for visual comparison.

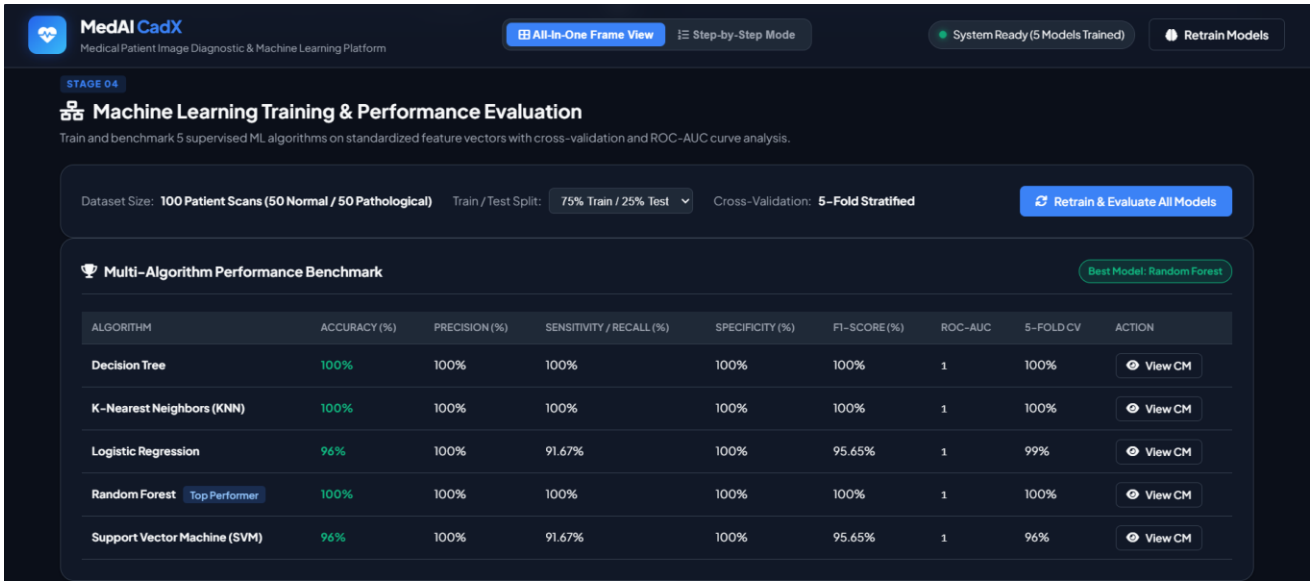


Fig 5: Stage 4 — Machine Learning Training and Performance Evaluation

The shown benchmark uses 100 patient scans, with 50 normal and 50 pathological cases, a 75% training / 25% testing split and 5-fold stratified cross-validation. Five supervised algorithms are compared. In the displayed run, Random Forest is marked as the best model.



Fig 6: Diagnostic Confusion Matrix and ROC-AUC Performance Curve

For the selected Random Forest model, the displayed confusion matrix contains 13 true negatives, 0 false positives, 0 false negatives and 12 true positives. The ROC-AUC panel compares the five trained models and includes a random-chance reference line.

Result:

A web-based medical patient image diagnostic and machine-learning platform was successfully demonstrated through the MedAI CadX interface. The workflow integrates patient scan loading, image preprocessing, radiomic feature extraction, multi-algorithm machine-learning evaluation, confusion-matrix analysis and ROC-AUC visualization in a single application.

In the displayed benchmark, the dataset contains 100 patient scans (50 normal and 50 pathological). Decision Tree, KNN and Random Forest show 100% test accuracy in the displayed run, while Logistic Regression and SVM show 96% accuracy. Random Forest is identified by the interface as the best model, with a displayed confusion matrix of TN=13, FP=0, FN=0 and TP=12. These values should be treated as the results of the shown benchmark run and should be revalidated when the dataset, split or training configuration changes.